

MeetCI: A Computational Intelligence Software Design Automation Framework

Igor Khokhlov, Chinmay Jain, Ben Miller-Jacobson, Andrew Heyman, Leonid Reznik, Robert St.Jacques
B. Thomas Golisano College of Computing and Information Sciences
Rochester Institute of Technology
Rochester, USA

ixk8996@rit.edu, caj4430@rit.edu, brm9994@rit.edu, ajh3126@rit.edu, lr@cs.rit.edu, rjsvcs@rit.edu

Abstract—Computational Intelligence (CI) algorithms/techniques are packaged in a variety of disparate frameworks/applications that all vary with respect to specific supported functionality and implementation decisions that drastically change performance. Developers looking to employ different CI techniques are faced with a series of trade-offs in selecting the appropriate library/framework. These include resource consumption, features, portability, interface complexity, ease of parallelization, etc. Considerations such as language compatibility and familiarity with a particular library make the choice of libraries even more difficult. The paper introduces MeetCI, an open source software framework for computational intelligence software design automation that facilitates the application design decisions and their software implementation process. MeetCI abstracts away specific framework details of CI techniques designed within a variety of libraries. This allows CI users to benefit from a variety of current frameworks without investigating the nuances of each library/framework. Using an XML file, developed in accordance with the specifications, the user can design a CI application generically, and utilize various CI software without having to redesign their entire technology stack. Switching between libraries in MeetCI is trivial and accessing the right library to satisfy a user's goals can be done easily and effectively. The paper discusses the framework's use in design of various applications. The design process is illustrated with four different examples from expert systems and machine learning domains, including the development of an expert system for security evaluation, two classification problems and a prediction problem with recurrent neural networks.

Keywords—Software design automation; computational intelligence techniques; computational intelligence libraries;

I. INTRODUCTION

Computational Intelligence (CI) is a field that is currently undergoing rapid technological development. The day-to-day “dumb” devices are being quickly replaced by their “intelligent” versions. The user's base for this technology has subsequently risen, as more and more developers research, test, and apply CI techniques. CI software has ballooned into numerous libraries and frameworks that allow programmers to access and design industry-specific applications.

While the CI term has been widely utilized in both academia and industry for a considerable duration, there is still no definition unanimously accepted by all communities. We refer to CI software as an umbrella term that includes the application of theories/concepts that employ methods and models analogous to those attributed to biological intelligent systems

and rely heavily on numerical data application [1]. Among such methodologies are artificial neural networks, fuzzy and learning systems, genetic algorithms and evolutionary computation, support vector machines and other machine learning techniques.

As new users enter the CI marketplace, deciding on the appropriate library can be a difficult, time consuming, and tedious task. Several libraries provide overlapping solutions, which can further blur the decision making process. Ultimately, once a framework is chosen, software is designed on an application-specific basis for an implementation and output. As this CI software base grows, many libraries can fall in-and-out of favor, being replaced for new functionality and breadth of services. Given that most of the code is written for a specific library, replacing said library requires almost a complete overhaul of the software, further delaying product development, increasing business cost and a further limiting CI adoption.

Evaluation and comparison of multiple implementations adds another layer of complexity when attempting to access CI techniques. Making intra-library comparisons between different CI algorithms is the least cumbersome approach because the user is not required to relearn a new framework; however, it is not always an option given in today's CI environment. CI frameworks' performance can drastically vary depending on the specific design by each framework [2]. In addition, certain frameworks can lack desired characteristics. When no single implementation supports all algorithms, the user must choose to eliminate one or more from their consideration, or consequently, support another library which requires the user to relearn and abstract away the different approaches and requirements of different libraries. This is even more typical of projects involving multiple implementations comparison of the same algorithm. Given these common, recurring problems, we have developed a framework to address these issues in the CI marketplace.

MeetCI, a software framework that promotes use of Computational Intelligence through software design automation is structured to be a generic tool. Users can select from a wide range of CI methodologies including applications of fuzzy rules systems, neural networks, and other machine learning techniques. The software framework is designated to become the first meeting point for a developer look-

ing to apply CI techniques and consider the wide selection of libraries/tools available. MeetCI is designed as an open source product under the MIT license and is available at <https://github.com/DataQualityLab/MeetCI>.

MeetCI starts with the user specifying their problem and an algorithm to solve it in an XML file. The framework extracts information from this file and trains a CI model ready to be used. At the highest level, MeetCI first determines whether the provided XML file describes an expert system or a machine learning problem, before continuing on to process the file in an appropriate way. Additionally, a provision has been created that allows a user to direct the framework for a solution from a specified library. If no selection is made, the framework uses default values and provides the user with a trained solution. MeetCI aims to support different benchmarking options to make these decisions simple.

The remaining paper is structured as follows: Section II provides an analysis of the current state of the CI field and related works. An explanation of the design decisions made by the team and system architecture is discussed in Section III and Section IV. In Section V, the XML specifications are provided and examined. Section VI provides examples, as well as the future work in this field.

II. COMPUTATIONAL INTELLIGENCE SOFTWARE: STATE OF THE FIELD

Over the last few decades, the CI community has investigated a plethora of methods and algorithms and developed a wide array of corresponding software to implement them. By now, many CI frameworks and libraries have emerged from both academia and industry. Some of them are rich in features, with limited performance and are more suited to research experiments, while some are performance oriented with production systems in mind. In an application design, the developer has to choose a framework most suited for their purposes and inevitably has to make some compromise to accommodate the limitations of the chosen framework. Detailing an exhaustive list of possible libraries and their capabilities with their advantages and disadvantages is outside of the scope of this paper. Such a list would need hundreds of entries, quite possibly thousands of implementations of an individual algorithm for a single language would be included. Instead, we briefly summarize some of the more popular computational intelligence libraries in Table I and Table II.

MeetCI started approximately 10 years ago, evolving from a simpler project originally focused on fuzzy expert system applications [21]. Since the MeetCI project began, a number of libraries and tools for computational intelligence have been developed that have some goals, features or design elements in common with MeetCI.

PyBrain [22], one of the libraries MeetCI currently makes use of, is a Python based library that supports many machine learning algorithm. One of the unique features of this library is the XML based tools that it provides. This can be found in the tools/customxml/ directory of the PyBrain folder. This tool assumes a high degree of familiarity with PyBrain and

essentially operates on the same level of abstraction as the API itself. Due to this fact, it is typically used only by professionals with a deep knowledge of this product.

KEEL, or Knowledge Extraction based on Evolutionary Learning [23], [24], [25], is a powerful tool in software engineering education, research and automation. The core focus of KEEL is to provide a single, stand-alone, fully integrated platform that can be used for research or education, primarily but not exclusively with evolutionary algorithms. KEEL features an extensive set of preprocessing capabilities and despite a stated focus on evolutionary methods, contains a wide list of algorithms of many types, for example a wide variety of fuzzy algorithms for regression, classification, clustering, etc [26].

TensorFlow [14] is one of the most popular open source libraries for numerical computation. This library is built on data flow graphs utilization, where the graph nodes represent mathematical operations. The graph edges represent the multidimensional data arrays, also known as tensors. TensorFlow allows the code execution on multiple CPUs and GPUs in a desktop, server or mobile device without code modification. It also includes a specialized toolkit for the data visualization.

The Microsoft Cognitive Toolkit (CNTK) [15] is another open source deep learning toolkit. Similar to TensorFlow, it describes neural networks as a series of computational steps via a directed graph. It allows a developer to easily implement various popular models such as feed-forward DNNs, convolutional neural networks, and recurrent neural networks. It also employs parallelization across multiple GPUs and servers.

CILib, or Computational Intelligence Library [27], [28] has some similarities with MeetCI. Both provide XML driven CI libraries, which allow a user to convert an XML description of a computational intelligence library into a working implementation. CILib is intended primarily for developing new algorithms using pre-existing components, and as such provides sophisticated capabilities for assembling hybrid or multi-part techniques simply by constructing an XML file.

Keras [16] a popular open-source library that pursues a similar goal as MeetCI. It is a python based deep learning library that acts as a wrapper and can work with Theano, TensorFlow, and CNTK. It allows easy and fast prototyping, supports convolutional networks and recurrent networks, and runs seamlessly on CPUs and GPUs. However, unlike MeetCI, Keras is limited to three libraries. It uses a Python API rather than XML or some other independent format to define the network to be implemented.

MeetCI is designated as a software design automation tool that simplifies the ability to produce applications utilizing CI techniques. It is intended to complement and be employed along with other CI software design frameworks, such as KEEL and CILib. The last ones are predominantly devoted to the CI algorithm design level, whereas MeetCI concentrates on implementation issues, including selecting the appropriate implementation library.

TABLE I
MACHINE LEARNING LIBRARIES COMPARISON: SAMPLE

	Learning Curve	Methods support		Performance	Big Data Support	Binding with other libraries
		Neural Network	Machine Learning			
Scikit [3]	Shallow	Medium	High	High	Yes	Yes
PyBrain [4]	Shallow	High	Low	Medium	Yes	Yes
Theano [5]	Shallow	High	No	High	Yes	Yes
FANN [6]	Steep	High	No	High	Yes	Yes
Rapid Miner [7]	Shallow	High	Medium	Medium	Yes	Yes
Oracle Data Miner [8]	Shallow	Low	High	Medium	Yes	Yes
Matlab [9]	Steep	High	High	Low	Yes	Yes
R Apps [10]	Medium	High	High	Low	Yes	Yes
Orange [11]	Medium	High	Medium	High	No	Yes
Java ML [12]	Steep	No	Medium	Medium	Yes	No
Neuroph [13]	Medium	High	No	Medium	No	No
TensorFlow [14]	Medium	High	No	High	Yes	Yes
Microsoft CNTK [15]	Steep	High	No	High	Yes	No
Keras [16]	Medium	High	No	High	Yes	Yes
Caffe [17]	Medium	High	No	High	Yes	Yes

TABLE II
EXPERT SYSTEM SHELL DEVELOPMENT TOOLS COMPARISON: SAMPLE

	Learning Curve	Feature Support	Performance	Security	Portability
Prolog	Shallow	Medium	Low	Medium	Low
CLIPS [18]	Steep	Medium	Low	High	Medium
JESS [19]	Medium	High	Low	High	High
Blaze Adviser [20]	Very High	Low	Medium	Very High	Very High

III. FRAMEWORK DESIGN GOALS

In the design of our framework, we aimed at improving our previous design and extending features found in other products. The design intends to improve:

- Extensibility by an opportunity to include new libraries and products;
- Maintainability by providing an option of switching between libraries and products;
- Interoperability between multiple language and product support;
- Ease of comparisons.

For a system of this sort, extensibility is vitally important as users may add support for whichever libraries and algorithms interest them. A focus on extensibility allows MeetCI to grow to better accommodate the needs of users over time without requiring substantial changes to the underlying framework.

MeetCI significantly improves the maintainability of computational intelligence software. Once a project is written using our system, switching between underlying libraries becomes trivial. If a library becomes unmaintained, switching to another library with an entirely different interface could be as simple as changing a single line of code.

The MeetCI design also makes it easier for users to incorporate libraries not written in the same language that the rest of their code uses. Though the core of our code and several of our supported libraries are written in Python, we

also provide support for machine learning libraries written in other languages, for example FANN (C/C++) [29] and Neuroph (Java). For the moment, MeetCI only features a Python interface. However, as the support expands in the future, MeetCI will be able to provide an API that would abstract this implementation from the end user.

MeetCI is also designed to facilitate comparisons between different libraries and techniques. Comparing multiple algorithms is slightly more complicated, but still only requires changing a few lines in the portion of the XML file that specifies the algorithm. In both cases, the behavior of the overall system remains essentially the same, so no user code needs change. This allows our system to be a powerful tool in comparing different implementations of an algorithm or different algorithms. In addition, it would make constructing automated tools to select a good algorithm for a particular problem fairly easy to write.

IV. FRAMEWORK ARCHITECTURE

The MeetCI architecture was designed to be easily accessible and extensible, modular and scalable. Its core functionality can be accessed via command line or API, depending on the user's preferences. Figure 1 provides a visual representation of the process of a trained model generation. First, the user must generate an XML file in accordance with the XML document structure discussed in Section V. Depending on the user's objectives outlined in the XML file, MeetCI validates and then returns an implementation model generated by an installed library. The user has the option to specify a desired library or rely on the default values. In addition, MeetCI supports the capability to compare performance of the same algorithm implementation with different libraries by providing speed and memory usage data. This allows for the best library determination for their algorithm and use case. The system maintains a live repository of installed libraries with their capabilities, which facilitates validation of library selections and provides the user with a list of supported options that meet the requirements defined in the XML file. To generate implementations, MeetCI uses a plugin system that abstracts

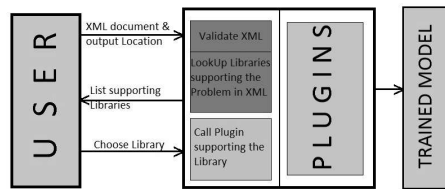


Fig. 1. MeetCI interacting with a user (training model stage).

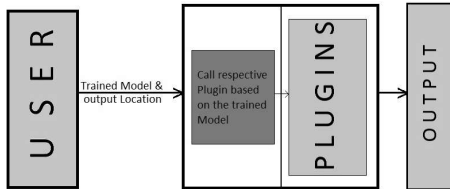


Fig. 2. MeetCI interacting with a user (using a trained model stage).

away intricacies between each library to provide a standardized interface.

Once an implementation is generated, it is stored persistently. Figure 2 demonstrates the framework's behavior after an implementation has been generated. At this point, the user is able to interface with the results through a provided script or user developed code.

MeetCI is split into two separate components internally, due to the differences between expert systems and machine learning. Though these components are conceptually similar, details of each implementation within MeetCI differ. The machine learning component uses an initial layer for basic validation and tracking current libraries/their functionality. Given a valid library selection, this layer passes the XML file to an adapter written in Python corresponding to the desired library. If the library is native to Python, this adapter will directly interface with the library. Otherwise, it will access another wrapper designed to easily interface with the additional library. See figures 3 and 4 for more information. The end result is a serialized trained model with a standard interface, which can be accessed from other software or queried directly from our provided testing script.

In the initial release of our project, we have chosen to support a restricted selection of algorithms as implemented in several libraries. The libraries were selected in large part because they are of the more widely used machine learning libraries, distinct but overlapping library support, several different languages represented and a variety of individual strengths. The currently available MeetCI version supports three libraries for Python based environment: Scikit [30], PyBrain [22] and Theano [31]. In addition, it supports Neuroph implemented in Java and FANN [29] in C/C++.

The expert system component is laid out in a conceptually similar way, but its specific implementation differs from the machine learning. Within the expert system section, the interpreter parses the XML document into a collection of objects designed to represent the facts and rules of the system. This collection is then passed to plugins that translate the objects

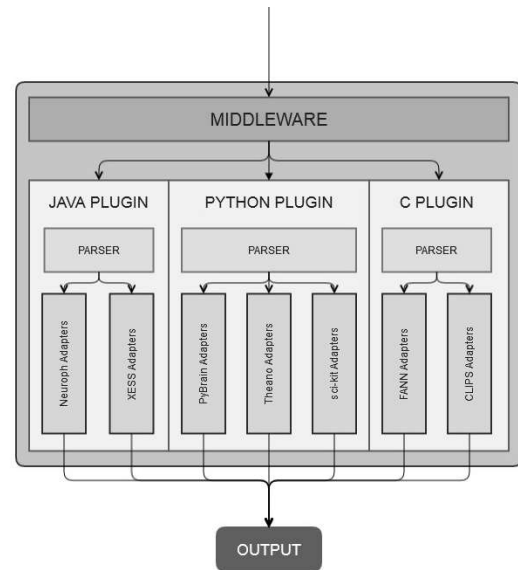


Fig. 3. MeetCI component diagram

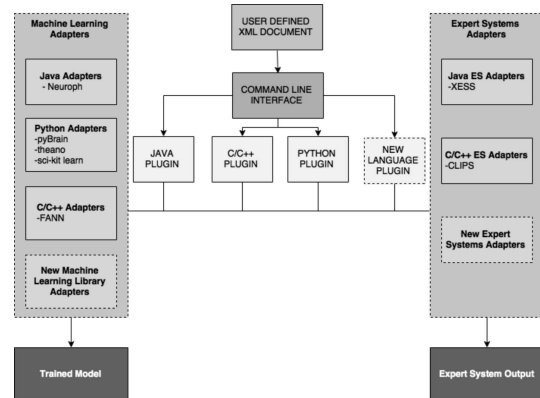


Fig. 4. MeetCI architecture design

into the native language of specific expert systems. The plugins are responsible for: a) providing input to the specific expert systems b) collecting feedback and c) making it available to the user through the interpreter API. Unlike the machine learning component, no serialization is considered necessary or desirable as a default behavior.

XML is a markup language and is described using tags [32]. This property allows XML files to be easily read and created. The accessibility of XML was a key tenet in our design choice. The elements in the XML file will be described based on an XML schema. Validations are done at a command line/API through the XSD schema definition. Basic validation includes checking certain conditions to ensure that the file has the correct type, the file is non empty, etc.

Once a problem statement is defined in an XML file, the software parses this file and retrieves information about the problem the user has defined (e.g. whether it is a classification problem, a prediction problem, or an expert system problem). Subsequently, MeetCI retrieves the algorithm name from the

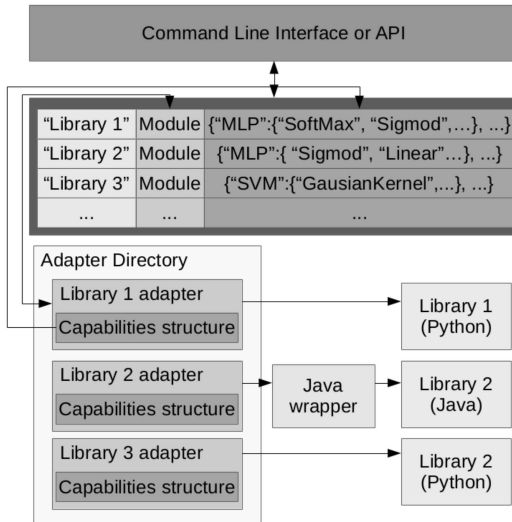


Fig. 5. Plug-in management system

algorithm tag, also within the XML file. Based on the particular problem on hand and algorithm name, the system examines the installed libraries that support this problem statement. If there is more than one library that supports the problem statement, the user is provided an option to choose the library.

Our XML structure allows for extensibility by allowing new algorithms simply by adding new "elementTypes" in the schema and utilizing them for some problem types. In addition, the XML design promotes ease of interface with custom software, as most programming languages provide support for XML conversion to objects and vice versa.

MeetCI supports plug and play architecture (see figure 5), which makes it easy to automatically detect new plugins and learn what capabilities they offer. To accomplish this, every time the system is initiated, the plugin directory contents are evaluated. For every *.py file located within the plugin directory, it generates an entry in a live repository, including a reference to the file which may be used to interact with its methods and variables, and a copy of a capabilities structure imported from the file. This structure is used to efficiently determine what installed libraries support a requested technique and allows MeetCI to conveniently call the methods of the selected plugin.

V. XML SPECIFICATION AND IMPLEMENTATION

The framework/library agnostic definition of a CI problem to solve is represented as an XML document. Each XML document captures the aspects of a problem as a hierarchy of elements. Top level XML tag choices determine the validity of future tags. Within MeetCI, the first element states if the problem is classified as <MachineLearning> or <ExpertSystem>.

These level restrictions are dictated by a schema, the specification for XML documents. The XML schema structures a CI problem as a tree of elements. The choice of elements at each stage represents the branching in the tree. Each XML

```

<MeetCI>
  <MachineLearning>
    <classification>
      <datafile>C:\JavaPlugin\Iris.csv</datafile>
      <input>4</input>
      <output>1</output>
      <classes>3</classes>
      <split>70</split>
      <delimiter></delimiter>
      <algorithm>
        <MultiLayerPerceptron>
          <inputLayerActivation>Linear</inputLayerActivation>
          <hiddenLayerActivation>Sigmoid</hiddenLayerActivation>
          <hiddenLayers>3,4</hiddenLayers>
          <outputLayerActivation>Sigmoid</outputLayerActivation>
          <momentum>0.01</momentum>
          <epochs>30</epochs>
          <learningRate>0.01</learningRate>
        </MultiLayerPerceptron>
      </algorithm>
    </classification>
  </MachineLearning>
</MeetCI>

```

Fig. 6. MeetCI XML document for solving the Iris problem [33] with a neural network model.

document define a CI problem, abiding to the restriction of the schema, would follow a specific path in this tree. The XSD definition is used to validate the XML document. Due to the inherent differences between machine learning and expert systems implementations, the XML specifications are broken out into two separate sections.

A. Machine Learning

Machine learning problems are split into different categories including <classification>, <prediction>, etc. This element must be included after using the <MachineLearning> element. For each category, there will be at least one <algorithm> element and unique set of child elements. Available <algorithm> elements include: <MultiLayerPerceptron>, <RadialBasisFunctionNetwork>, <RecurrentNeuralNetwork>, <RandomForest>, <SupportVectorMachine>.

The choice of algorithm is restricted based on the category in the upper level. It is also possible for the same algorithm to occur under more than one category. Each algorithm element has its own set of unique parameters as its child elements. Figure 6 presents a sample XML document representing a single instance of a CI problem.

As described earlier, the document represents a path in the specification tree. Here the document begins with the root <MeetCI> tag, which common to all, and branches out into MachineLearning in the next level. The values of the elements under MultiLayerPerceptron (momentum, epochs, etc) and Classification (input, delimiter) are specific to the problem.

B. Expert Systems

Expert Systems module contains the knowledge base of the system represented as Facts and Rules. Rules are used to check a condition and execute an action. Fact, action and clause are abstract types, which are extended by concrete types. The XML document only contains the concrete types.

The *fact* base type is extended by:

- <predicate> - a name and a value,
- <struct> - defines a related collection of variables, each containing a unique name, type, and default value within the scope of the <struct>

- `<instance>` - a named instantiation of a `<struct>` with specific values for each of the internal variables

The *clause* base type is extended by many sub-clauses including: `<equal>`, `<notEqual>`, `<greaterThan>`, `<greaterThanOrEqual>`, `<lessThan>`, `<lessThanOrEqual>`.

The *action* base type is extended by:

- `<set>` - used to update the value of predicates, specific fields inside an instance or even the value of arguments passed to a rule
- `<runRule>` - used to execute another rule within the system.

The XML document is primarily used by the Validation module and the individual Adapters in the system. In both cases the system, using existing libraries, generates a class tree based on the XSD schema and parses individual XML documents into an object tree. Given the XML and XSD, the object tree should match the type of classes generated from the XSD. This restriction is used by the validation module. In the adapter level, the parsed objects are used to extract information about the problem, which is used to run the algorithms using the chosen library.

Every algorithm has an XML schema defined in an XSD file. The user defines an XML file based on this XSD file. Given the XSD and XML file, the system uses tools to generate data structures (e.g. class definitions) specific to the schema. For Python, we are using generateDS, while for java we are using JAXB, which generates a file with class definitions based on XSD. One benefit of using an XSD schema is that most of the validations can be done by defining a proper structure for the XML file, which is used for executing the algorithm. Every library has an adapter file that imports a file that contains data structures (for e.g. class definitions) generated using xml schema. We then use a parser to load xml data to the data structures. These data structures can then be used to provide input to the algorithm specific to that library. Once we train the system for classification or prediction, we then take the trained model and serialize it to a pickled data structure, which the user can then import to test on other datasets.

VI. IMPLEMENTATION EXAMPLES

MeetCI is designed to facilitate simple and easy CI software development. The most challenging part is writing the XML file, as per the specifications discussed in the above section. Once the XML file with the problem definition is ready, feeding it into the system and extracting the solution out of it is taken care by the software. The XML can be specified to the software by a simple command line as `python main.py -f XML_filename`.

The framework's main component is written in Python. The only command line argument required is the XML file name with the complete path of the file. After executing this command the framework will ask the user for their library preferences by listing all the installed and supported libraries in a list format. After the user enters the library name, the software displays the complete XML in its raw format and

TABLE III
RESULTS REPRESENTATION IN THE PYBRAIN'S AND NEUROPH'S MLP

Total Error	xx.xxx
Training Accuracy	xx.xx%
Total Error	

proceeds with its execution. Finally, after the model is trained as per the specification mentioned in the XML file, the training results are passed to the user. The trained model is saved at an output location specified by the user. The following examples are given to explain the framework's functionality and its interaction with a user.

A. Problem 1: Iris Problem

The Iris problem [33] is one of the most widely used testing examples. It is a classification problem that consists of four inputs and one output. The aim of this problem is to identify the exact species of the Iris flower in contention. The Iris can be one of three varieties: Iris setosa, Iris virginica and Iris versicolor. The three varieties differ in the length and width of the flower. So the four inputs to the model are the length and width of the sepal and the petal. And output is the class of the input that is the variety name. The dataset consist of 150 instances and is divided 3:1 ratio for training and testing purposes. All of the instances are complete and the dataset has a large spread of values.

This problem can be solved using a Multi-Layer Perceptron model or a Radial Basis Model amongst few other algorithms. These algorithms are implemented by almost all the machine learning libraries available today. Now using the XML file defined in section 5 and using MeetCI we will find a solution to the Iris problem in the following steps:

- 1) Feed the XML to MeetCI using the command as - `python main.py -f MLP_Iris.xml`
- 2) The system will ask the user about their library preference as- Please type the name of the library which you would like to use for execution: Suppose the user types pyBrain and clicks on Enter.
- 3) The software will implement the problem using Py-Brain's MLP implementation and return the training results to the user as shown in Table III.
- 4) The user can access the trained model from the output directory.
- 5) To depict the advantages of MeetCI, we repeat step 1 and modify step 2 as follow Please type the name of the library which you would like to use for execution: Suppose the user types Neuroph and clicks on Enter.
- 6) The software will implement the problem using Neuroph's MLP implementation and return the training results to the user as shown in Table III.
- 7) Now the user can access the new trained model from the output directory.
- 8) Similarly step 2 can be modified to train neural network models for Iris problem across different libraries.

B. Problem 2: MNIST Problem

MNIST [34], [35] is another classic test case of a classification problem, which can be solved using machine learning. The MNIST dataset we used consists of 60,000 instances for training and 10,000 instances for testing. Each instance consists of images of the size 28 x 28 pixels and represents single digit from zero to nine from the numeric system. Each pixel carries color code between 0 and 255 to differentiate between black and white respectively. The dataset is pickled, as it is easy to use it that way in Python. The model is trained with the training data and then validated by the validation data before we begin testing. The aim of the experiment is to recognize the correct digit based upon the information available in the input image instance.

MeetCI includes a demo for solving MNIST problem using Theano library and the Multi-Layer Perceptron algorithm.

C. Problem 3: Speech Recognition Problem

Speech Recognition [33] is used for implementing Recurrent Neural Networks. It's time series data with 8800 instances. Each instance consists of an audio file which represents a digit being spoken. Every audio file is represented as a set of 13 attributes. The dataset is complete and has no missing values. This problem solution is demonstrated using Recurrent Neural Network capabilities of the PyBrain library. The software supports only one implementation of this problem.

D. Problem 4: Expert system for security evaluation (ESSA)

The ESSA [21] defines a knowledge that can be used to evaluate the current state of security controls, policies, and procedures in a given environment when provided with user input describing those factors. Five control fields were identified from the National Institute of Standards and Technology (NIST) guidelines regarding computer security. These control fields included: risk management, security controls, authorized processing, system security plan, and personnel security. The ESSA contains rules evaluating specific criteria in each of these control fields, and determines an overall rating for each field. Finally an overall security assessment rating is produced by aggregating the individual ratings for all of the control fields.

VII. CONCLUSION

This paper introduced MeetCI, a software design automation tool for the computational intelligence field. It is designed to facilitate language agnostic applications using the constant growing list of CI tools and libraries entering the marketplace. This project primarily focuses on the CI implementation, not CI algorithm design. Currently, MeetCI supports a wide breadth of libraries implementing various CI methods including: fuzzy logic, expert systems, various artificial neural networks models including recurrent ones, support vector machines, random forests, and other machine learning techniques.

The framework's design focuses on solving some of the main issues with utilization of current CI libraries. MeetCI allows developers to quickly resolve any library migration

issues, by allowing specific library-level decisions to be made within the XML document, instead of going through a direct interface with each specific library. Evaluation of multiple CI implementations across a variety of libraries requires editing a single line within the XML document. This does not only provide quick and easy comparisons, but multiple implementations require only one core XML file. Finally, as the CI field continues to grow, extensibility was put at the core of MeetCI's design. Adding new programming languages, as well as specific machine learning/expert systems libraries, has been embedded into the structure of MeetCI. As new libraries are created, interleaving their functionality into MeetCI is a straightforward process. Library implementations can be easily abstracted away and the user interface requires very little to no additional changes. Finally, the XML structure and corresponding schema have been designed to supplement extensibility features. As algorithms are created and refined, the user defined XML can be easily updated to reflect these developments. XML's ubiquitous support across every major programming language allows it to serve as a strong foundation for CI solution design, as well as ability to work across a variety of platforms, both new and old.

As discussed, MeetCI allows for solving the same problem using various tools. Benchmarking is paramount when working with different implementations. The framework allows for benchmarking to be done in many ways, each implementation can be compared with respect to its accuracy, time performance, resource utilization and many more parameters. Furthermore, having a powerful user interface to show these benchmarking results will significantly improve the user experience. From the educational perspective, benchmarking would help new students understand the differences between implementations across different CI platforms. Students can use these benchmarking techniques and practically understand the polarities between the various CI techniques and tools. Benchmarking between the same implementations across different CI libraries and tools is another facet that can help the user.

MeetCI is intended to be the first meeting point for developers looking to apply CI techniques and the wide array of current libraries. Focused on software design automation, users looking to apply computational intelligence algorithms can use MeetCI to access a variety of existing libraries easily, without having to focus on a specific language and framework. Furthermore, MeetCI has the ability to promote CI to the education and developer community. Removing key hurdles for accessing various CI tools and libraries, this framework can be used in the educational setting to demonstrate the power of CI to new users without teaching a single language/framework. MeetCI is designed to promote CI techniques to new users who want to apply them in real world situations. Within professional settings, developers can use MeetCI to overcome business concerns including the support of a specific framework and migration issues that can ground new CI projects to a halt. MeetCI serves as a multifaceted solution to the existing concerns of the CI marketplace and is designed to be dynamic

enough to adjust with this rapidly changing field.

MeetCI is designed as an open source product under the MIT license. The current framework version is available for download along with the problem examples and datasets at <https://github.com/DataQualityLab/MeetCI>.

ACKNOWLEDGMENT

This material is partially based upon work supported by the National Science Foundation under Award # ACI-1547301 and National Security Agency under Award # H98230-I7-I-0200.

REFERENCES

- [1] L. Reznik, "General principles and purposes of computational intelligence in systems science and cybernetics," *Systems Science and Cybernetics-Volume III*, p. 198.
- [2] S. Nissen. (2003) Fann 6 - benchmarks. Accessed on 27.12.2017. [Online]. Available: <http://fann.sourceforge.net/report/node8.html>
- [3] (2017) scikit-learn: machine learning in python scikit-learn 0.19.1 documentation. Accessed on 27.12.2017. [Online]. Available: <http://scikit-learn.org/stable/>
- [4] (2010) Pybrain. Accessed on 27.12.2017. [Online]. Available: <http://pybrain.org/>
- [5] (2017) Welcome theano 1.0.0 documentation. Accessed on 27.12.2017. [Online]. Available: <http://deeplearning.net/software/theano/>
- [6] (2017) Fast artificial neural network library (fann). Accessed on 27.12.2017. [Online]. Available: <http://leenissen.dk/fann/wp/>
- [7] (2017) Rapidminer - #1 open source predictive analytics platform. Accessed on 27.12.2017. [Online]. Available: <https://rapidminer.com/>
- [8] (2017) Oracle data mining. Accessed on 27.12.2017. [Online]. Available: <http://www.oracle.com/technetwork/database/options/advanced-analytics/odm/index.html>
- [9] (2017) Machine learning with matlab. Accessed on 27.12.2017. [Online]. Available: <https://www.mathworks.com/solutions/machine-learning.html>
- [10] (2017) R: The r project for statistical computing. Accessed on 27.12.2017. [Online]. Available: <https://www.r-project.org/>
- [11] G. Shaulsky, F. Borondics, and R. Bellazzi. (2017) Orange data mining fruitful & fun. Accessed on 27.12.2017. [Online]. Available: <http://orange.biolab.si/>
- [12] T. Abeel. (2012) Java machine learning library (java-ml). Accessed on 27.12.2017. [Online]. Available: <http://java-ml.sourceforge.net/>
- [13] (2008) Java neural network framework neuroph. Accessed on 27.12.2017. [Online]. Available: <http://neuroph.sourceforge.net>
- [14] (2017) <https://www.tensorflow.org/>. Accessed on 03.01.2018. [Online]. Available: <http://deeplearning.net/tutorial/gettingstarted.html>
- [15] (2017) Microsoft cognitive toolkit — microsoft docs. Accessed on 03.01.2018. [Online]. Available: <https://docs.microsoft.com/en-us/cognitive-toolkit/>
- [16] F. Chollet *et al.*, "Keras," <https://github.com/keras-team/keras>, 2015.
- [17] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," *arXiv preprint arXiv:1408.5093*, 2014.
- [18] (2017) Clips: A tool for building expert systems. Accessed on 27.12.2017. [Online]. Available: <http://clipsrules.sourceforge.net>
- [19] E. Friedman-Hill. (2013) Jess, the rule engine for the java platform. Accessed on 27.12.2017. [Online]. Available: <http://herzberg.ca.sandia.gov>
- [20] (2017) Fico blaze advisor decision rules management system - fico. Accessed on 27.12.2017. [Online]. Available: <http://www.fico.com/en/products/fico-blaze-advisor-decision-rules-management-system#overview>
- [21] L. Reznik and R. S. Jacques, "Fuzzy expert system shell development with computer security assessment application," in *Fuzzy Systems Conference, 2007. FUZZ-IEEE 2007. IEEE International*. IEEE, 2007, pp. 1–6.
- [22] T. Schaul, J. Bayer, D. Wierstra, Y. Sun, M. Felder, F. Sehnke, T. RČ1/4ckstieČ, and J. Schmidhuber, "Pybrain," *Journal of Machine Learning Research*, vol. 11, no. Feb, pp. 743–746, 2010.
- [23] I. Triguero, S. González, J. M. Moyano, S. García, J. Alcalá-Fdez, J. Luengo, A. Fernández, M. J. del Jesús, L. Sánchez, and F. Herrera, "Keel 3.0: an open source software for multi-stage analysis in data mining," *International Journal of Computational Intelligence Systems*, vol. 10, no. 1, pp. 1238–1249, 2017.
- [24] J. Alcalá-Fdez, L. Sanchez, S. Garcia, M. J. del Jesus, S. Ventura, J. M. Garrell, J. Otero, C. Romero, J. Bacardit, V. M. Rivas *et al.*, "Keel: a software tool to assess evolutionary algorithms for data mining problems," *Soft Computing-A Fusion of Foundations, Methodologies and Applications*, vol. 13, no. 3, pp. 307–318, 2009.
- [25] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, L. Sánchez, and F. Herrera, "Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework," *Journal of Multiple-Valued Logic & Soft Computing*, vol. 17, 2011.
- [26] J. Alcalá-Fdez and J. M. Alonso, "A survey of fuzzy systems software: Taxonomy, current research trends, and prospects," *IEEE Transactions on Fuzzy Systems*, vol. 24, no. 1, pp. 40–56, 2016.
- [27] G. Pampara, A. P. Engelbrecht, and T. Cloete, "Cilib: A collaborative framework for computational intelligence algorithms-part i," in *Neural Networks, 2008. IJCNN 2008.(IEEE World Congress on Computational Intelligence)*. IEEE International Joint Conference on. IEEE, 2008, pp. 1750–1757.
- [28] T. Cloete, A. P. Engelbrecht, and G. Pampara, "Cilib: A collaborative framework for computational intelligence algorithms-part ii," in *Neural Networks, 2008. IJCNN 2008.(IEEE World Congress on Computational Intelligence)*. IEEE International Joint Conference on. IEEE, 2008, pp. 1764–1773.
- [29] S. Nissen, "Implementation of a fast artificial neural network library (fann)," *Report, Department of Computer Science University of Copenhagen (DIKU)*, vol. 31, p. 29, 2003.
- [30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, no. Oct, pp. 2825–2830, 2011.
- [31] J. Bergstra, O. Breuleux, P. Lamblin, R. Pascanu, O. Delalleau, G. Desjardins, I. Goodfellow, A. Bergeron, Y. Bengio, and P. Kaelbling, "Theano: Deep learning on gpus with python," 2011.
- [32] (2015) Extensible markup language (xml). Accessed on 27.12.2017. [Online]. Available: <https://www.w3.org/XML/>
- [33] M. Lichman. (2013) UCI machine learning repository. Accessed on 27.12.2017. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [34] Y. LeCun, C. Cortes, and C. Burges. (2010) Mnist handwritten digit database. Accessed on 27.12.2017. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [35] (2015) Getting started & deeplearning 0.1 documentation. Accessed on 27.12.2017. [Online]. Available: <http://deeplearning.net/tutorial/gettingstarted.html>